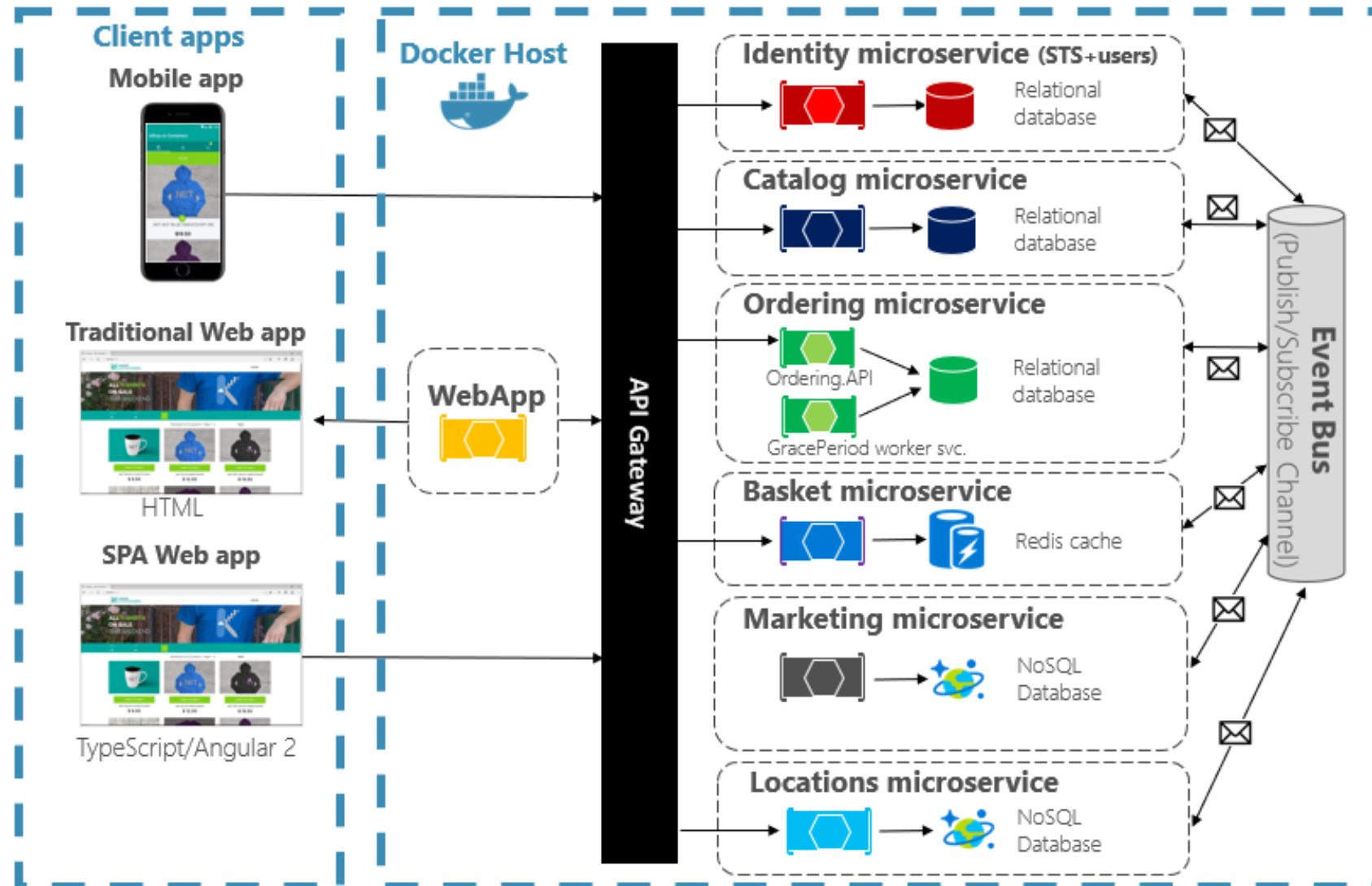


Developer Operations (devops)

Dependency Management

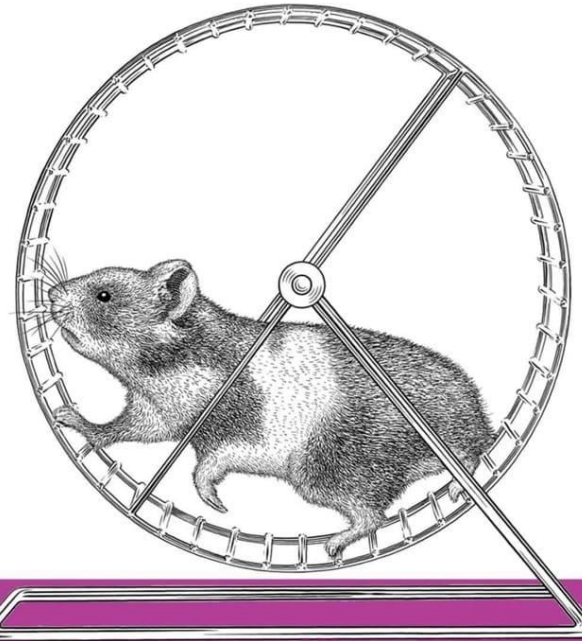


Challenge?



Newest Book?

“What did I do to deserve this?”



Resolving Broken Dependencies

This is Your Life Now

ORLY?

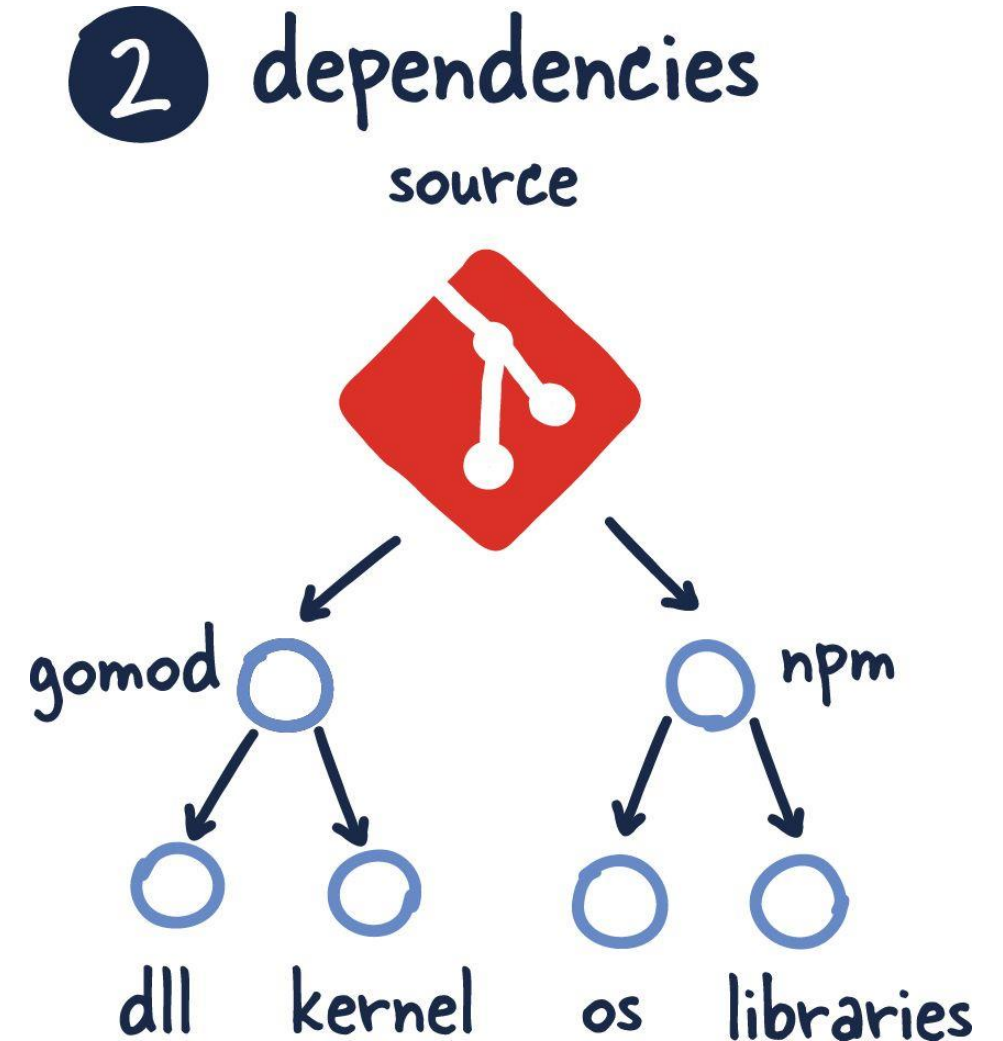
@ThePracticalDev

Clear access from sourcecode base

- **A twelve-factor app never relies on implicit existence of system-wide packages.**

It declares all dependencies, completely and exactly, via a *dependency declaration* manifest.

- [...]
- Twelve-factor apps also do not rely on the implicit existence of any system tools.





Which dependency management frameworks do you know?

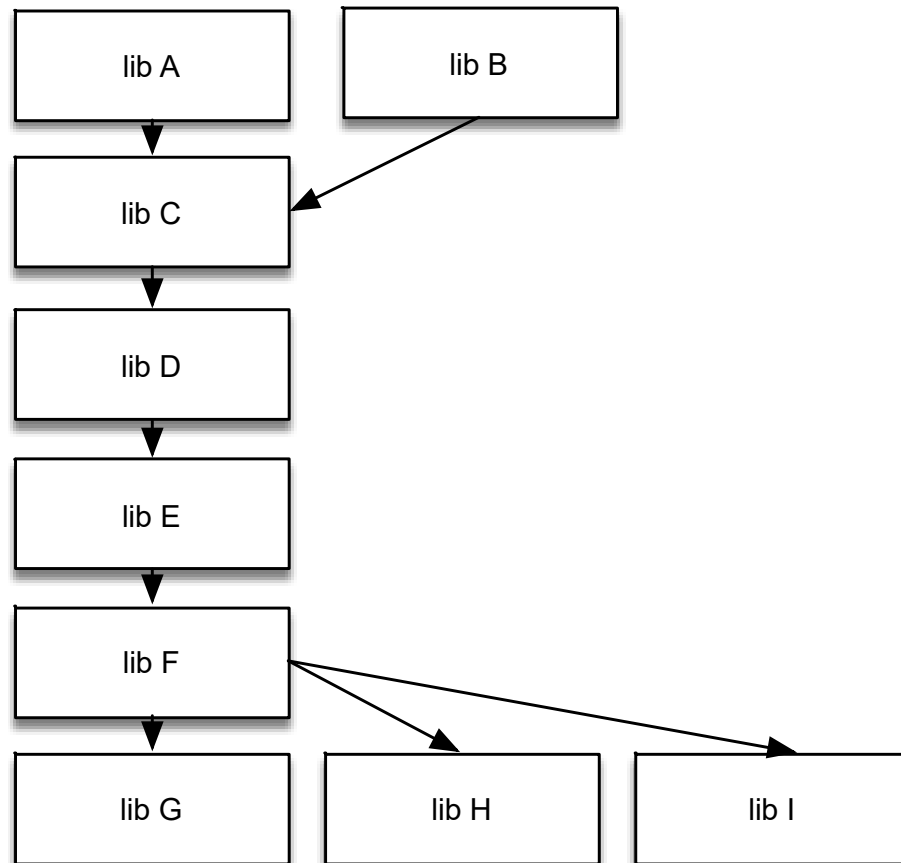
① Start presenting to display the poll results on this slide.

Dependency Trees

→ **buildpack-spring** **git:(main)** ./mvnw dependency:tree | wc -l
101

→ **buildpack-quarkus** **git:(main)** ./mvnw dependency:tree | wc -l
190

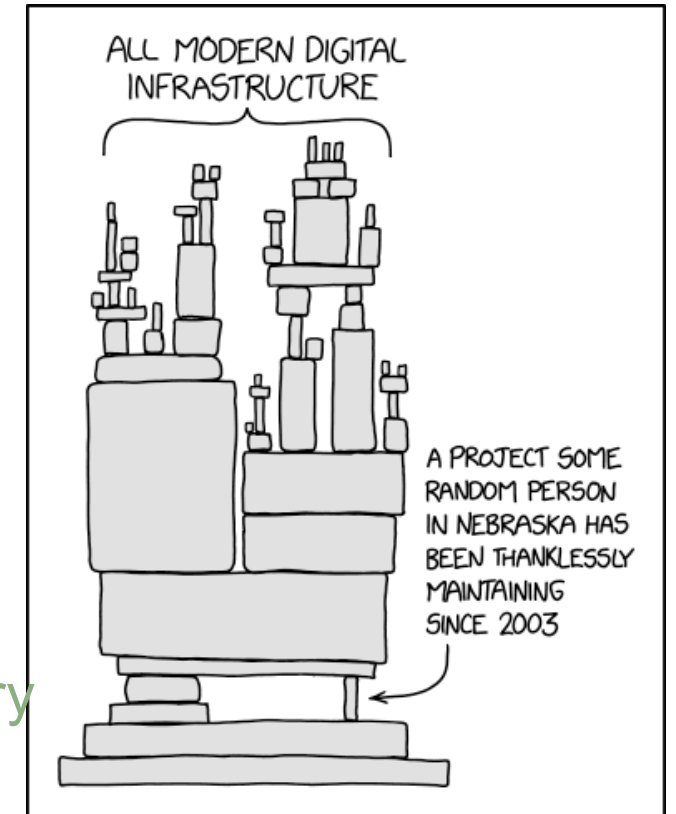
Large Dependency Trees



Long dependency chains

- + Intransparent
- + Potentially inconsistent
- + Potentially unreliable
- + Potentially unsecure
- + Potentially unlicensed

- Review installed packages
- Scan transitive packages regularly with specific tools
- Cache transitive packages

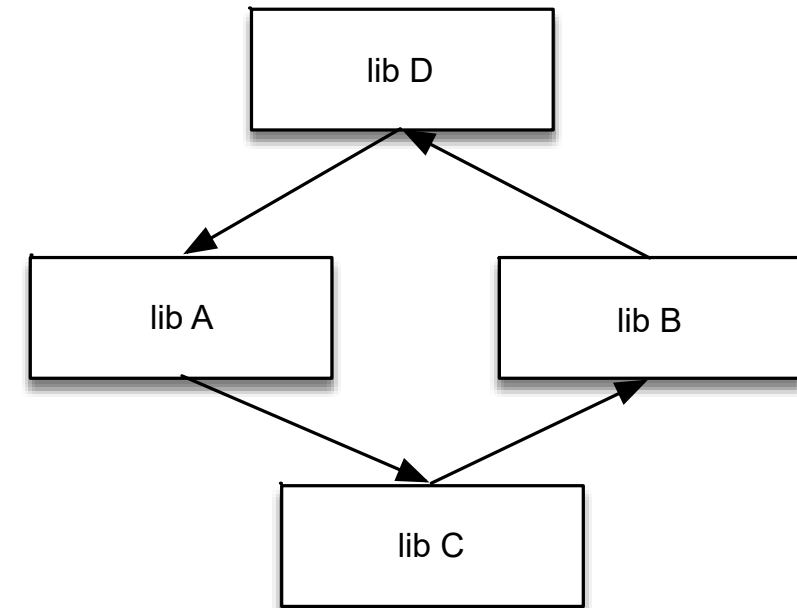
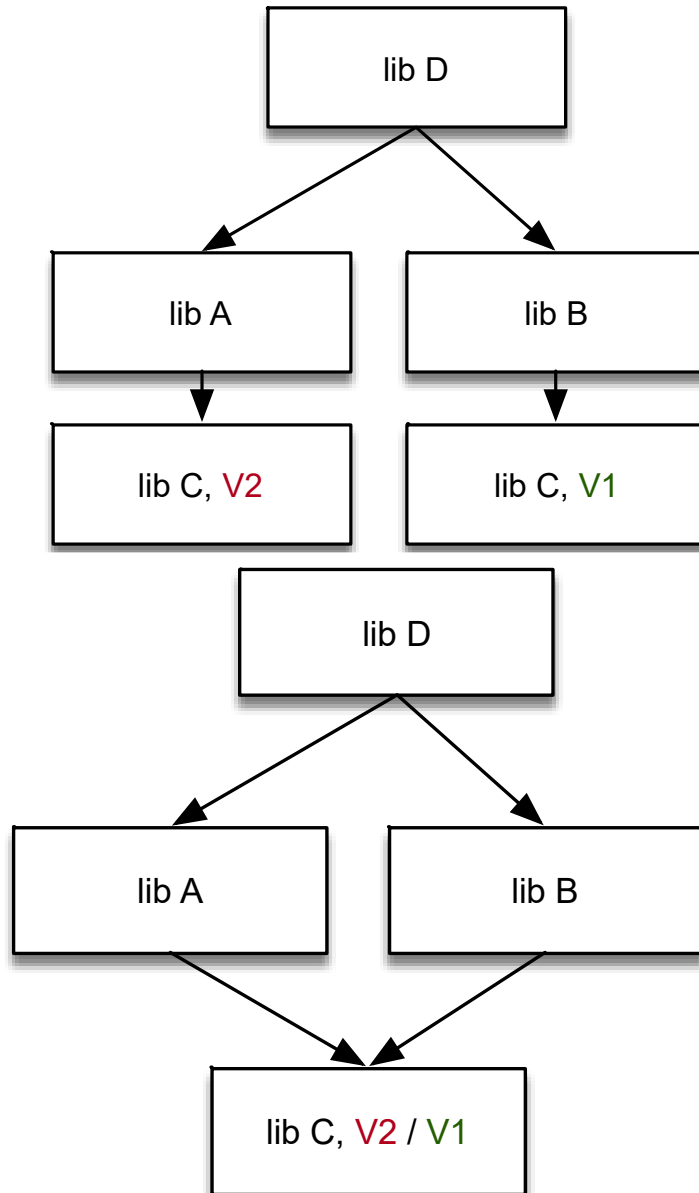


<https://xkcd.com/2347/>

Further problems of dependency management

Conflicting Versions in dependencies / Diamond dependency / Circular dependencies

- + Unclear resolution-strategies (based on tool)
- + Shortest path?
- + Parallel installation?
- + Complex environments with duplicate dependencies in different versions



- Review dependency graph
- Using Sandboxing for dependencies

Build Time Analysis

Idea: Building Artifact according to Maven Lifecycle



Lifecycle Stage3	Non-Cached	Cached
mvn validate	3.3sec	0.18sec
mvn compile	4.9sec	0.55sec
mvn test	4.0sec	2.533sec
mvn package	4.7sec	2.7sec
mvn verify	2.68sec	2.72sec
mvn install	3.29sec	2.833sec

<https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>

Software Bills of Material

“An SBOM is formal and machine-readable metadata that uniquely identifies a software package and its contents; it may include other information about its contents, including copyrights and license data. SBOMs are designed to be shared across organizations and are particularly helpful at providing transparency of components delivered by participants in a software supply chain.”

- Linux Foundation

Sample label for
Macaroni & Cheese

① **Start Here** →

② **Check Calories**

③ **Limit these Nutrients**

④ **Get Enough of these Nutrients**

⑤ **Footnote**

Nutrition Facts	
Serving Size 1 cup (228g)	
Servings Per Container 2	
Amount Per Serving	
Calories 250	Calories from Fat 110
	% Daily Value*
Total Fat 12g	18%
Saturated Fat 3g	15%
Trans Fat 3g	
Cholesterol 30mg	10%
Sodium 470mg	20%
Total Carbohydrate 31g	10%
Dietary Fiber 0g	0%
Sugars 5g	
Protein 5g	
Vitamin A	4%
Vitamin C	2%
Calcium	20%
Iron	4%

* Percent Daily Values are based on a 2,000 calorie diet. Your Daily Values may be higher or lower depending on your calorie needs.

	Calories	2,000	2,500
Total Fat	Less than	65g	80g
Sat Fat	Less than	20g	25g
Cholesterol	Less than	300mg	300mg
Sodium	Less than	2,400mg	2,400mg
Total Carbohydrate		300g	375g
Dietary Fiber		25g	30g

⑥ **Quick Guide to % DV**

- 5% or less is Low
- 20% or more is High

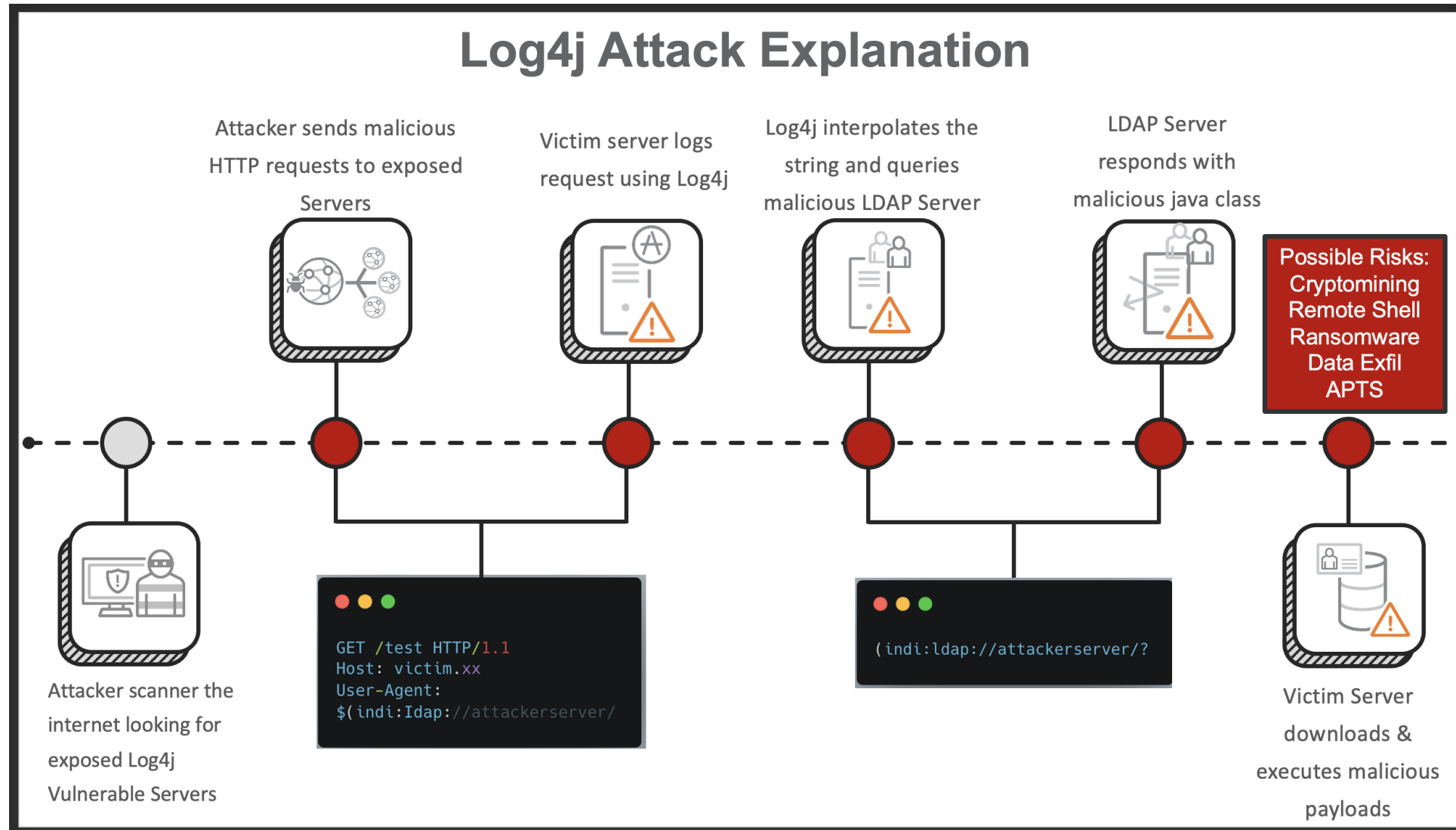
Example SBOM, SPDX

- ##### Package: github.com/gin-gonic/gin
-
-
- PackageName: github.com/gin-gonic/gin
- SPDXID: SPDXRef-Package-go-module-github.com-gin-gonic-gin-0eb9e28dbb25e10b
- PackageVersion: v1.10.0
- PackageSupplier: NOASSERTION
- PackageDownloadLocation: NOASSERTION
- FilesAnalyzed: false
- PackageChecksum: SHA256: 9d3bb285ad5362a81e773cadb0a62a9daf837cba2ce3a9d3bf6ca0172f3a1c55
- PackageSourceInfo: acquired package info from go module information: /myapp
- PackageLicenseConcluded: NOASSERTION
- PackageLicenseDeclared: NOASSERTION
- PackageCopyrightText: NOASSERTION
- ExternalRef: SECURITY cpe23Type cpe:2.3:a:gin-gonic:gin:v1.10.0:*:*:*:*:*
- ExternalRef: SECURITY cpe23Type cpe:2.3:a:gin_gonic:gin:v1.10.0:*:*:*:*:*
- ExternalRef: SECURITY cpe23Type cpe:2.3:a:gin:gin:v1.10.0:*:*:*:*:*
- ExternalRef: PACKAGE-MANAGER purl pkg:golang/github.com/gin-gonic/gin@v1.10.0

Handling SBOMs

DEMO

Why? Supply Chain Attacks / Log4Shell



Why? Supply Chain Attacks / XZ-Backdoor



Andres FreundTec

@AndresFreundTec@mastodon.social

I was doing some micro-benchmarking at the time, needed to quiesce the system to reduce noise. Saw sshd processes were using a surprising amount of CPU, despite immediately failing because of wrong usernames etc. Profiled sshd, showing lots of cpu time in liblzma, with perf unable to attribute it to a symbol. Got suspicious. Recalled that I had seen an odd valgrind complaint in automated testing of postgres, a few weeks earlier, after package updates.

Really required a lot of coincidences.

XZ Outbreak (CVE-2024-3094)



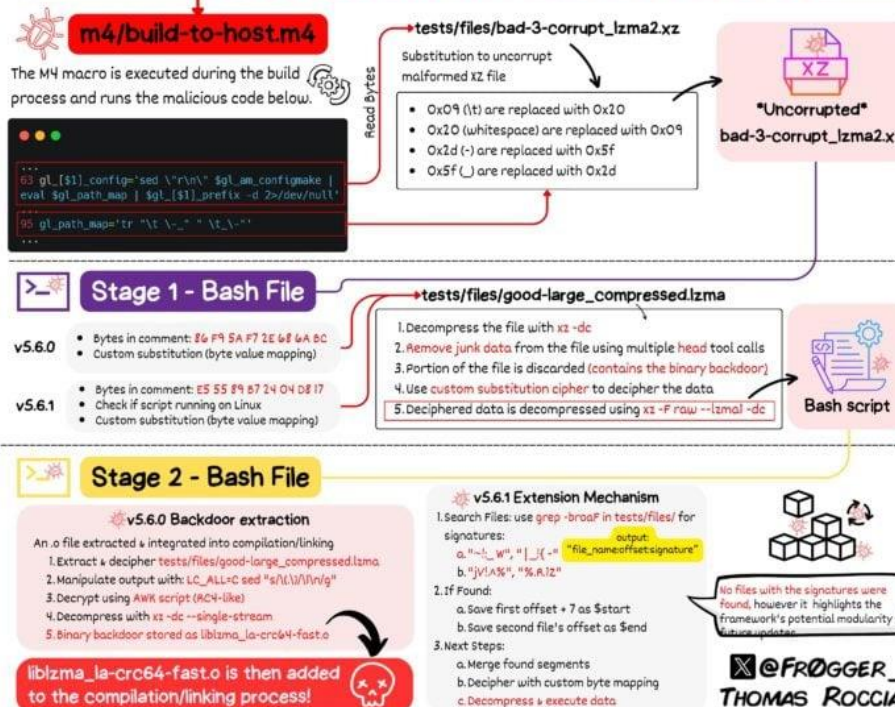
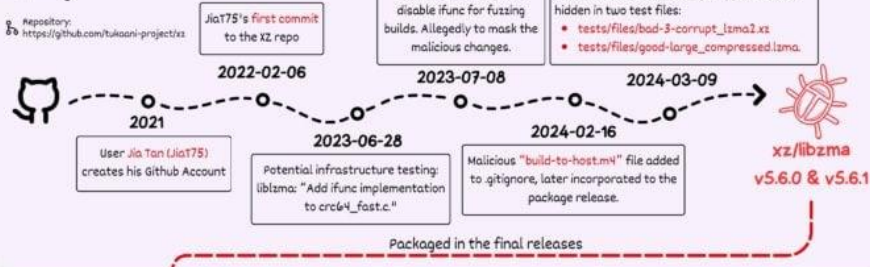
XZ Utils is a collection of open-source tools and libraries for the XZ compression format, that are used for high compression ratios with support for multiple compression algorithms, notably LZMA2.



On Friday 29th of March, Andres Freund (principal software engineer at Microsoft) emailed oss-security informing the community of the discovery of a backdoor in xz/libzma version 5.6.0 and 5.6.1.



Github Activity Summary (user: JiaT75)



<https://www.logpoint.com/en/blog/emerging-threats/xz-utils-backdoor>

Why? Supply Chain Attacks / npm Libraries Compromised

- **Scale & reach:** 2.6B weekly downloads
- **Foundational dependencies:** The affected packages are utility libraries
- **Stealthy payload:** The injected code wrapped browser APIs, silently rewriting cryptocurrency wallet transactions before signing.

All affected packages:

- ansi-styles@6.2.2
- debug@4.4.2
- chalk@5.6.1
- supports-color@10.2.1
- strip-ansi@7.1.1
- ansi-regex@6.2.1
- wrap-ansi@9.0.1
- color-convert@3.1.1
- color-name@2.0.1
- is-arrayish@0.3.3
- slice-ansi@7.1.1
- color@5.0.1
- color-string@2.1.1
- simple-swizzle@0.2.3
- supports-hyperlinks@4.1.1
- has-ansi@6.0.1
- chalk-template@1.1.1
- backslash@0.2.1

 Open (RESOLVED) Version 4.4.2 published to npm is compromised #1005
 MetaMask/meta... #1342

08 Sep 2025 17:35 CEST

Hello, thanks. Actually found out about this on bluesky.

Yes, I've been pwned. First time for everything, I suppose. It was a 2FA reset email that looked shockingly authentic. I should have paid better attention, but it slipped past me. Sincerely sorry, this is embarrassing.

- I've been locked out of my account on `npm`. I'm awaiting support's response to me. If someone at NPM is able to get in contact with me to escalate, ticket number is 3738263.
- NPM is only affected. It was a personal account. Repositories are not affected.
- The email came from `support at npmjs dot help`.

<https://github.com/debug-js/debug/issues/1005#issuecomment-3266868187>

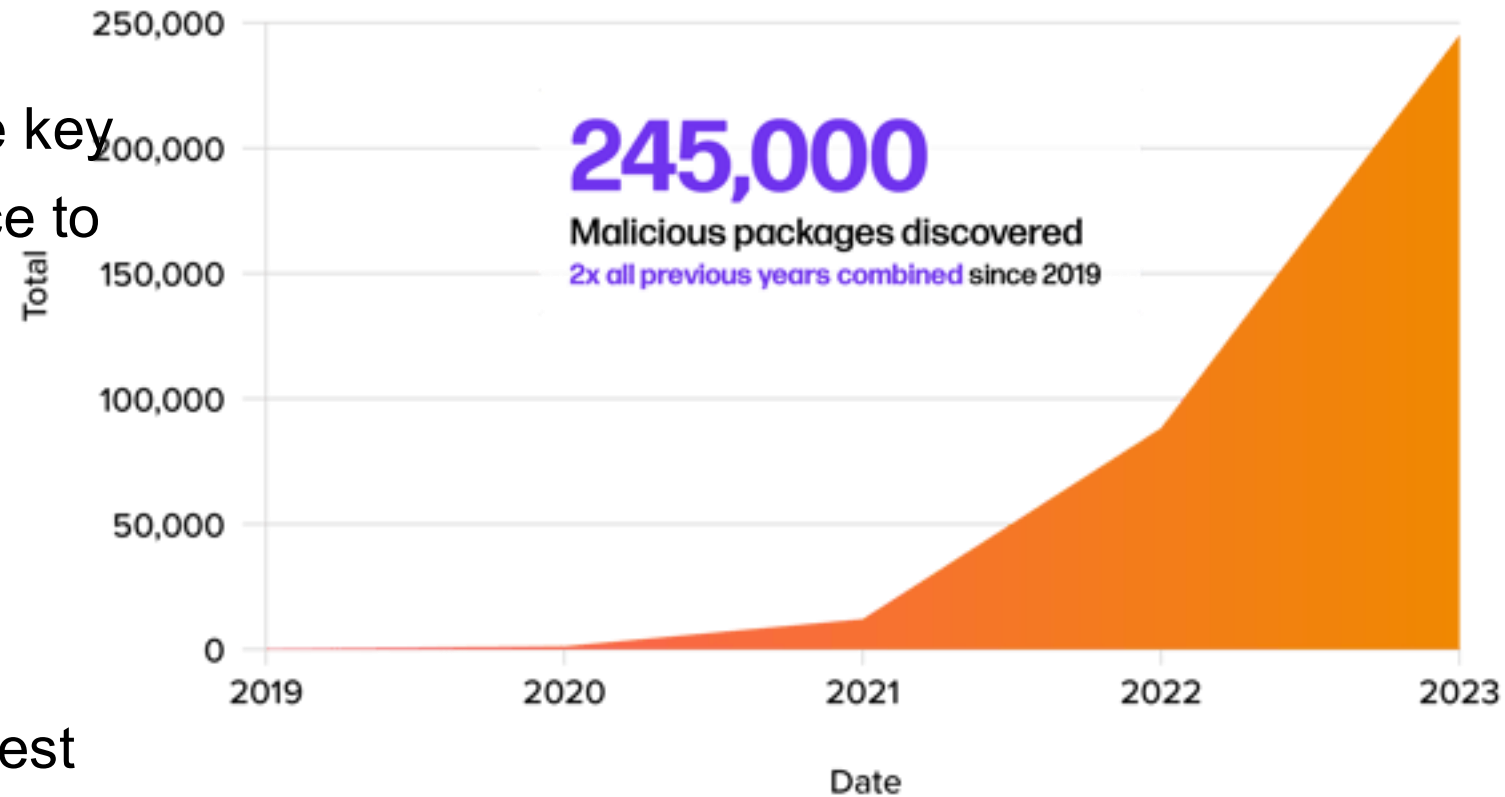
Qualys Analysis Blog Post: <https://t.ly/2Tlnj>

Supply Chain Attacks

- Numbers of (transitive) dependencies barely manageable
- In case of an incident, transparency is the key
- Nearly 13% of log4j downloads still reference to log4shell vulnerable versions
 - Builds still running....
 - ...whereas sources are not adapted

→ Supply Chain Attacks are one of the biggest security risks when building software

NEXT GENERATION SOFTWARE SUPPLY CHAIN ATTACKS (2019–2023)



Vulnerabilities of Software

Is stripping down all dependencies the best way?

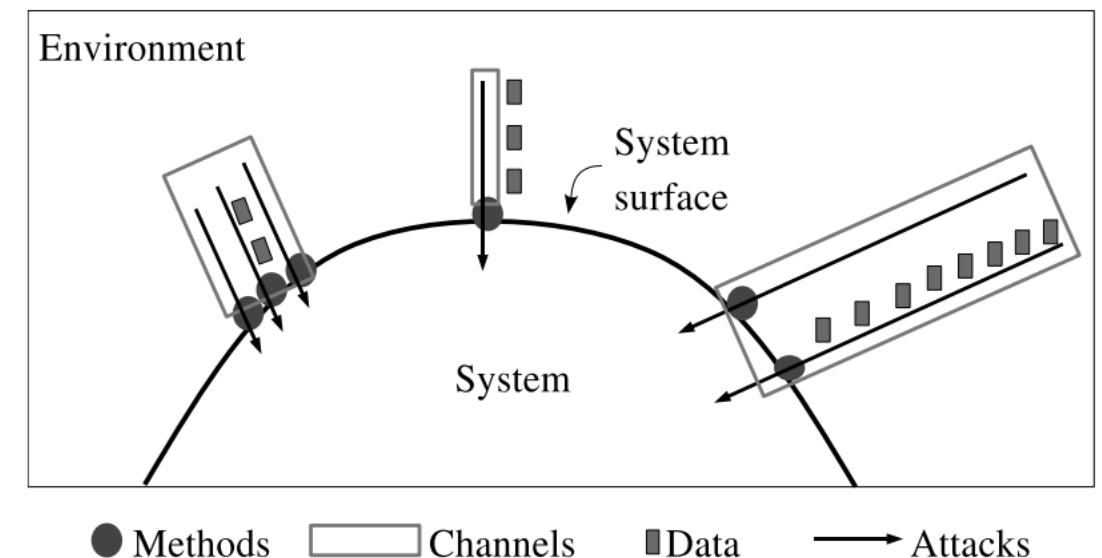
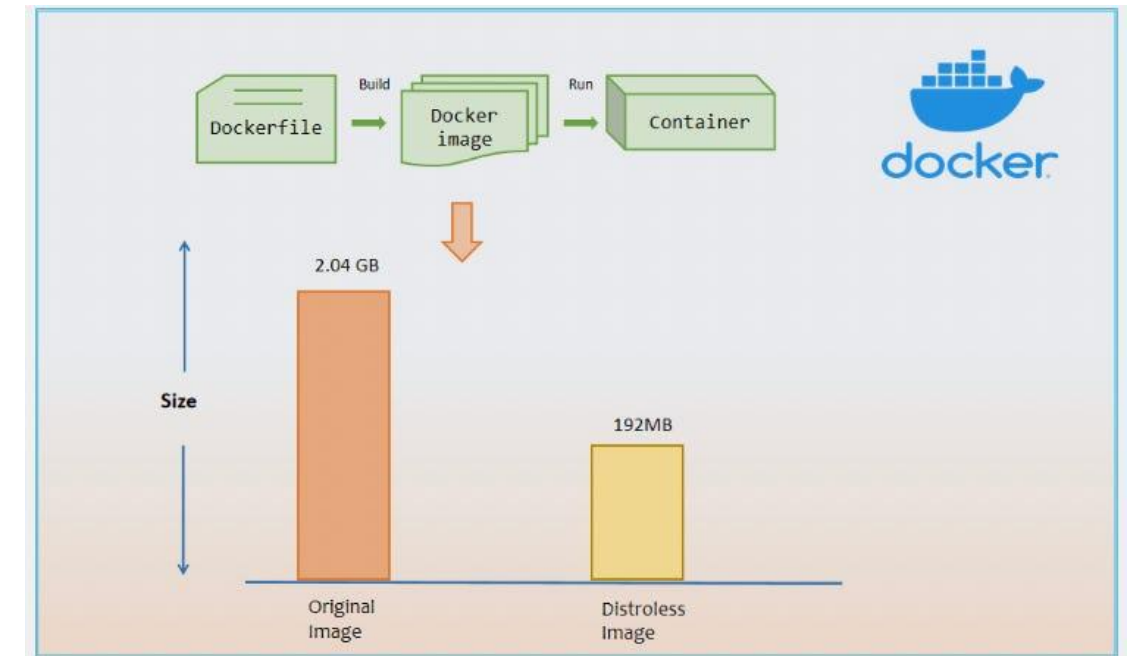
- The more libraries, the more possible vulnerabilities
 - aim to remove duplicate, multiversed, transitive dependencies
- Distroless to the rescue?
 - No shell, no package mgmt

But:

- Size != attack surface
 - What is important, is the execution path of a process in the system
 - Attacks can not make use directly of any files available
- There is always an operating system

Links:

- <https://www.redhat.com/en/blog/why-distroless-containers-arent-security-solution-you-think-they-are>
- <http://reports-archive.adm.cs.cmu.edu/anon/2008/CMU-CS-08-152.pdf>



Tools



grype

<https://github.com/anchore/grype>



syft

<https://github.com/anchore/syft>



<https://github.com/aquasecurity/trivy>

Silent Updates [0-xmin] / Homework

Think of a scenario, where an unattended / silent update of a dependency can happen:

